ePACS Offline Installer

Engineering Guide

Patterns, Pitfalls, Conventions, and Playbooks for the Development Team

Version: 1.0

Audience: ePACS Engineers (current and onboarding)

Date: May 2026

Owner: NLPSV Engineering — ePACS Installer Team

COMPANION DOCUMENTS

Solution Architecture & Design — ePACS_SAD_v1.2.docx

Implementation Plan — ePACS_Offline_Installer_Plan v3.5

Source: docs/engineering-guide.md

Table of Contents

How to Read This Guide

This document is organized for two audiences: engineers writing new code and engineers debugging an incident. Each section is self-contained — you can jump straight to a topic you need. Sections 1 to 10 mirror the source engineering-guide.md, expanded with sequence diagrams, decision trees, and concrete code examples. Sections 11 to 18 add the operational backbone — coding standards, branch and CI/CD, local development, code review, debugging playbooks, performance budget, security review, and onboarding — that the source guide left implicit.

> If you read only one thing — read Section 19 (Critical Rules). Every rule there is enforced by an analyzer, a CI gate, or a code-review checklist. Violations break the build.

Quick Reference Card

Top Ten Rules (laminate this page)

#	Rule
1	Never hardcode paths, ports, thresholds, or credentials. Everything from configuration.
2	Every DELETE must write to sync_outbox in the same transaction (before-state captured).
3	Every amendment must include reason + approver (configurable enforcement).
4	AUTO_INCREMENT seed = pacsid × 10^{10} — never manually set AUTO_INCREMENT.
5	Every state transition writes a checkpoint (fsync'd, write-then-rename).
6	PII never appears in logs — use [Sensitive], [DoNotLog], [Mask] attributes.
7	Heartbeat / sync failures never block business operations — fire-and-forget with circuit breaker.
8	Schema migrations classified by DDL algorithm — COPY on > 1M rows uses pt-online-schema-change.
9	Backup before every destructive operation — mandatory, not optional.
10	Test with power-cut simulation — every long operation must survive hard shutdown.

Where things live

Concern	Project / Path
State machine, mode detection	Installer.Core.StateMachine
Schema fingerprint, drift detection	Installer.Core.SchemaMigration / Schema
Upgrade engine (side-by-side, junction)	Installer.Core.Upgrade
Prechecks (OS, disk, RAM, port, GPO)	Installer.Actions.Prechecks
Service registration, ACLs, configs	Installer.Actions.Install
Uninstall with Override Token gating	Installer.Actions.Uninstall
Health, drift, log rotation, disk monitor	Installer.Agent.Monitors
File sync (attachments → NLDR)	Installer.Agent.FileSync
Heartbeat to CoopsIndia Dashboard	Installer.Agent.Heartbeat
Outbox drain (MySQL → Kafka)	Sync.Agent.Outbox / Outbox.Relay
Inbox processing + conflict resolution	Sync.Agent.Inbox
Connectivity circuit breaker	Sync.Agent.Connectivity
Reconciliation (nightly + on-demand)	Sync.Agent.Reconciliation
NLDR HTTP transport	Sync.Transport.Http

Concern	Project / Path
Backup / restore engine	BackupRestore
Manifest parse + signature + hash + ATA	ManifestVerifier
Support bundle with redaction	SupportBundle
Contracts, options, security primitives, errors	SharedKernel
Sequence-tables inventory	/config/sequence-tables.yaml
Error catalog	/config/error-catalog/installer.yaml
DDL migrations	/migrations/V###__*.sql, /migrations/R__*.sql

1. AUTO_INCREMENT Seeding and ID Space Partitioning

1.1 Technical query

How do we prevent AUTO_INCREMENT primary key collisions between (a) multiple offline PACS nodes generating records independently, (b) NLDR (central) generating records that flow down to PACS, and (c) records syncing from PACS to NLDR — without re-keying?

1.2 Analysis

The DDL (docs/AP_DDL.sql) reveals a range-partitioned BIGINT ID space. The massive AUTO_INCREMENT values you see in production are not sequential counters — they are PACS-prefixed composite IDs.

Evidence from the schema (production AP):

Table	AUTO_INCREMENT
ln_applicationmain	21,152,377,172,115,393
fa_ledger	133,779,293,356,240
ln_productpurposemapping	2,209,997,707,094,314,824

Supporting infrastructure:

- sequences table — per-PACS, per-table sequence counters (pacsid, sequencename, nextsequenceid)
- sequencetables table — registry of tables using sequence-based IDs
- pacsserialnumbers table — PACS-specific human-readable serial registry

Three ID-related columns on ~646 business tables:

Column	Type	Purpose
PRIMARY KEY (SlNo, ApplicationNo, ...)	BIGINT AUTO_INCREMENT	Globally unique — seeded per-PACS
idgeneratorforpacs	BIGINT	PACS-local sequence counter for offline generation
SerialNumberOfPacs	LONGTEXT	Human-readable serial (vouchers, receipts)
SourceId	TINYINT	Origin: 1=local PACS, 2=from NLDR, 3=legacy

1.3 Solution

Formula: seed = pacsid × RANGE_SIZE where RANGE_SIZE = 10^{10} (10 billion IDs per PACS per table).

```
BIGINT ID Space (0 to 9.2 × 10¹⁸):

[NLDR Reserved      :    1 — 9,999,999,999]
[PACS 2115          : 21,150,000,000,000 — 21,159,999,999,999]
[PACS 2116          : 21,160,000,000,000 — 21,169,999,999,999]
[PACS 3042          : 30,420,000,000,000 — 30,429,999,999,999]
...
```

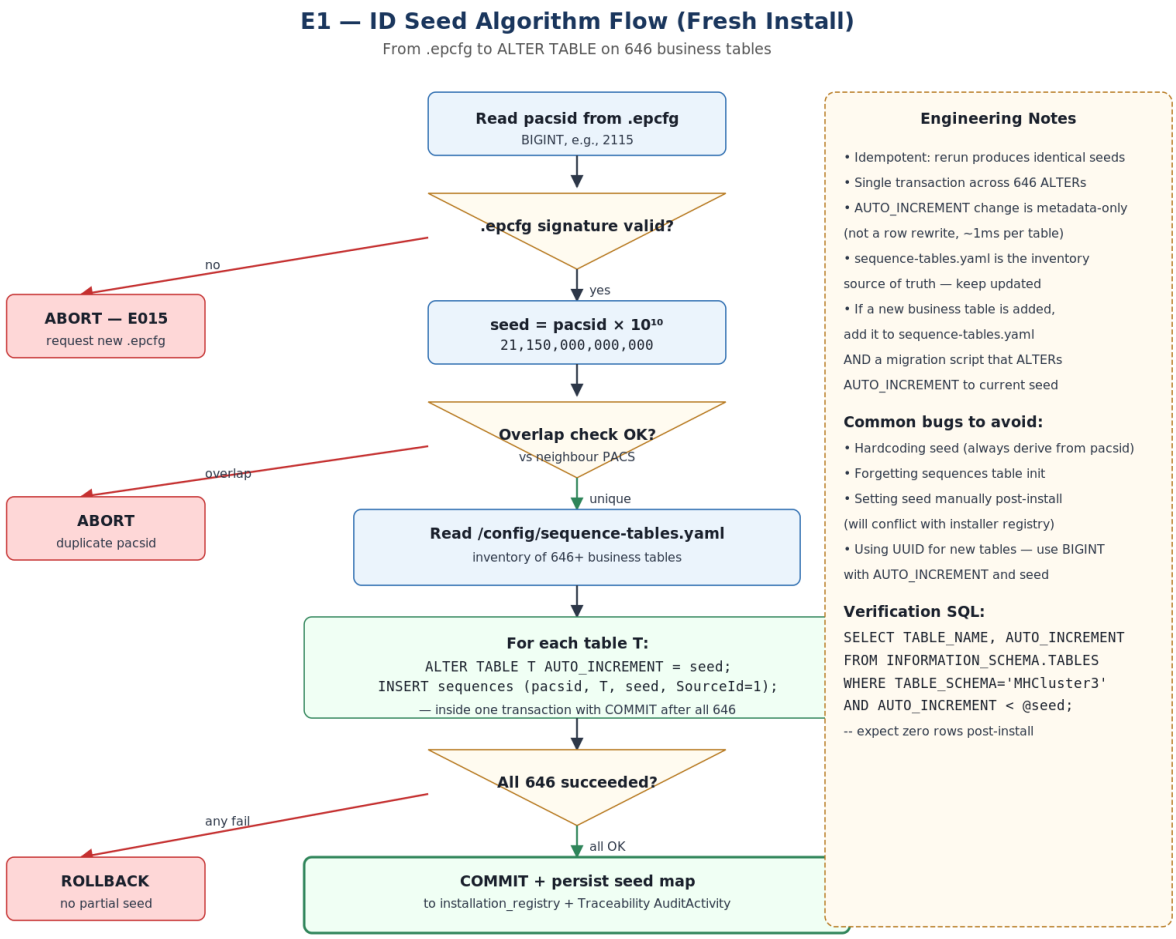

Figure: *ID Seed Algorithm Flow — From .epcfg to ALTER TABLE on 646 business tables*

Installer actions during fresh install

- **1.** Read pacsid from .epcfg (BIGINT, e.g., 2115).
- **2.** Compute seed = pacsid × 10,000,000,000.
- **3.** ALTER TABLE <t> AUTO_INCREMENT = <seed> for all 646+ business tables (single transaction).
- **4.** Initialize sequences with nextsequenceid = seed per table.
- **5.** All locally-created records get SourceId = 1.

1.4 Mitigation

- Installer Agent monitors AUTO_INCREMENT values daily against BIGINT ceiling (9.2×10^{18}).
- Alert at 50 % usage; block upgrades at 75 %.
- Currently only ln_productpurposemapping is at 24 % — all others well under 1 %.

1.5 Pitfalls

Pitfall	Impact	Prevention
Two PACS assigned same	ID collision, data corruption	Validate uniqueness during .epcfg generation; installer

Pitfall	Impact	Prevention
pacsid		checks against state registry
RANGE_SIZE too small	PACS exhausts its range	10 billion IDs per table is effectively infinite for a single PACS (~300+ years at 100 records/sec)
NLDR uses IDs in PACS range	Collision on inbound sync	NLDR restricted to range 1—9,999,999,999; enforced by application layer
sequences out of sync with AUTO_INCREMENT	Duplicate key errors	Installer verifies sequences.nextsequenceid >= MAX(PK) on every upgrade
Legacy data migration with conflicting IDs	Collision with new records	Legacy records use SourceId = 3; migrated with original IDs preserved in NLDR range
Hardcoding seed (any value)	Site-specific bugs that pass tests	Roslyn analyzer rejects literal AUTO_INCREMENT in code; only via /config/sequence-tables.yaml

1.6 Additional considerations

- Restore to new machine: AUTO_INCREMENT is part of the MySQL datadir. Restoring a backup preserves the correct seed. No re-seeding needed.
- Schema fingerprint: The seed value is NOT part of the schema fingerprint (it's data, not structure). Drift detection ignores AUTO_INCREMENT values.
- Sync idempotency: The globally-unique PK serves as the natural idempotency key for sync. If NLDR receives a record with a PK it already has, it's a duplicate.

2. Deletion Handling and Sync Consistency

2.1 Technical query

- Hard deletes (voucher deletion, PDS / Trading deletions) that remove data permanently.
- In-place amendments (auditor corrections) that modify already-synced financial data.
- Bulk deletions via Correction Tool.
- NLDR becoming stale when PACS deletes / amends synced records.

2.2 Analysis

Current state from docs/deletionsenerio.md:

- Hard deletes: implemented (physical DELETE from tables).
- Soft deletes: not implemented.
- Data versioning: not implemented.
- Maker-checker for amendments: not implemented.
- Audit trail for deletions: partial (6 deletion-pattern tables exist).

Deletion-pattern tables across modules:

Module	Tables
FAS	fa_voucherdeletionmain, fa_voucherdeletiondetails
PDS	pds_purchasedelete, pds_salesdelete
Trading	tr_purchasedelete, tr_salesdelete
Correction Tool	CorrectionTool_LoansActivityLog, CorrectionTool_MembershipActivityLog

Voucher deletion flow today:

- Data saved to temp tables (fa_vouchermaintemp).
- User posts → moves to permanent tables (fa_vouchermain).
- User deletes → logs to deletion tables → hard DELETEs from temp tables.

2.3 Solution

> "Nothing is ever truly deleted from the sync perspective." Deletions and amendments are EVENTS that must be propagated to NLDR. Every business mutation that may have already synced to NLDR — INSERT, UPDATE, DELETE, AMENDMENT — produces an outbox event in the same MySQL transaction as the mutation itself.

```
// Pattern — runs inside a single MySQL transaction
BEGIN TRANSACTION
  DELETE FROM fa_vouchermaintemp WHERE VoucherNo = ?;

  INSERT INTO sync_outbox (
    event_type, change_type, entity_type, entity_id,
    payload_json, before_state_json, payload_hash
  ) VALUES (
    'DATA_CHANGE', 'DELETE', 'fa_voucher', <voucher_id>,
    NULL, <before_state_json>, SHA2(<payload>, 256)
```

```
 );
COMMIT;
```

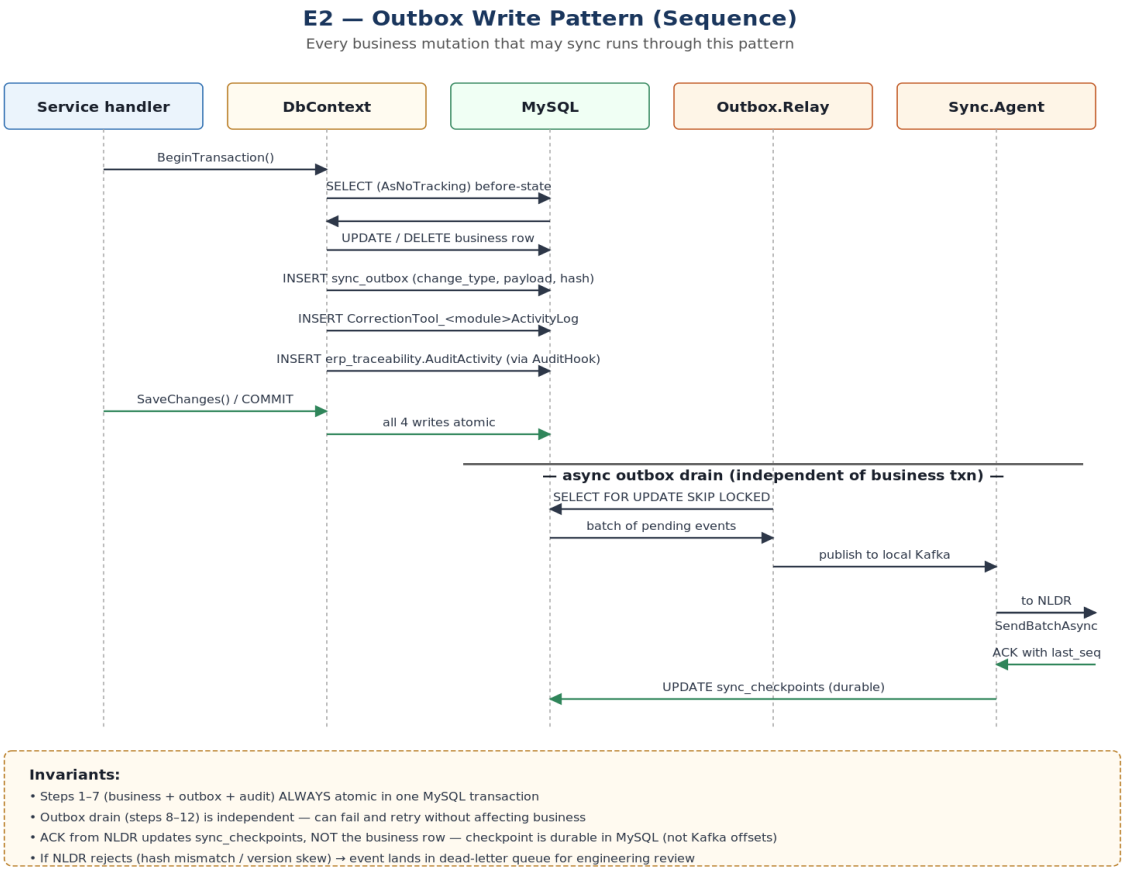

Figure: *Outbox Write Pattern (Sequence) — Every business mutation runs through this*

change_type values in sync_outbox

change_type	Meaning
INSERT	New record created (existing behavior)
UPDATE	Record modified (payload = after-state)
DELETE	Record removed (before_state_json = the deleted data)
AMENDMENT	Auditor correction (payload = before + after + reason + approver)

NLDR-side action by change_type

change_type	NLDR action
INSERT	Create record (idempotent — check event_id first)
UPDATE	Update record (idempotent — version check via payload_hash)
DELETE	Soft-delete on NLDR side (mark is_deleted=1, retain for audit)
AMENDMENT	Update record + insert amendment_history row; if backdated, flag for audit

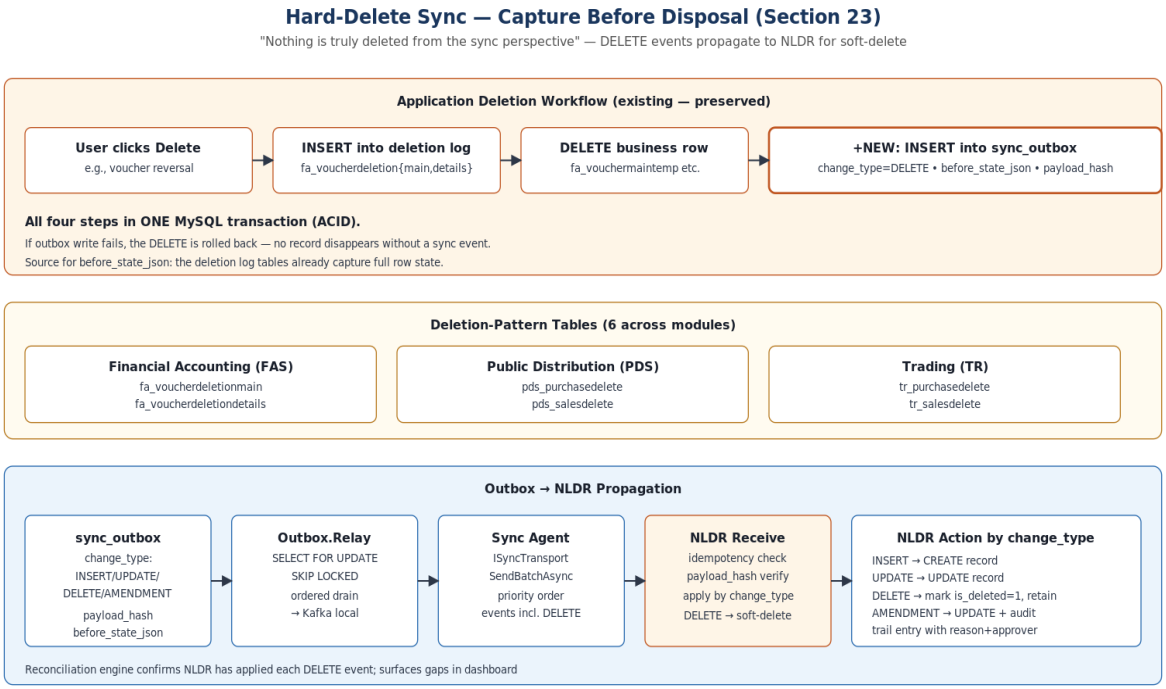

Figure: *Hard-Delete Sync — Capture Before Disposal*

2.4 Mitigation

Risk	Mitigation
Deletion without outbox entry (application bug)	Reconciliation detects missing records (nightly comparison)
Bulk deletion exceeds safety threshold	BulkDeleteThreshold (default 10); above threshold requires mandatory backup
Backdated correction on synced data	AMENDMENT event carries before+after; NLDR applies and flags for audit
Accidental bulk deletion	CorrectionTool log + sync outbox DELETE events; restore from pre-op backup
Amendment without reason/approver	AmendmentRequiresReason=true, AmendmentRequiresApprover=true (config)

2.5 Pitfalls

Pitfall	Impact	Prevention
DELETE trigger fires but transaction rolls back	Orphan sync event	Use application-level capture (same transaction), not triggers
Before-state JSON exceeds outbox column size	Sync event lost	Use LONGTEXT for before_state_json; compress if > 1 MB
Parent DELETE with FK-cascade children	Cascade not captured individually	Capture parent deletion; NLDR cascades on its side
Amendment to non-synced	Unnecessary AMENDMENT	Check sync_status before creating AMENDMENT; if never

Pitfall	Impact	Prevention
record (post-sync)	event	synced, treat as INSERT
Correction Tool bypasses outbox	NLDR never learns of the correction	Code review + analyzer + CI check enforce outbox writes

2.6 Future enhancement (v2): soft delete

For v2, add soft-delete columns to critical financial tables (additive, ALGORITHM=INSTANT, zero downtime):

```
ALTER TABLE fa_vouchermain
  ADD COLUMN is_deleted TINYINT(1) NOT NULL DEFAULT 0,
  ADD COLUMN deleted_at DATETIME NULL,
  ADD COLUMN deleted_by VARCHAR(100) NULL,
  ADD COLUMN deletion_reason VARCHAR(500) NULL,
  ALGORITHM=INSTANT;
```

3. DDL Drift and Schema Migration

3.1 Technical query

- Schema modifications made outside the installer (manual ALTER TABLE in the field).
- Failed partial migrations (power-cut mid-migration).
- Irreversible DDL (column drops, type changes) that prevent rollback.
- 1,057 tables with complex FK dependencies during migration.

3.2 Analysis

Schema profile from docs/AP_DDL.sql:

- 1,057 tables, 81 foreign keys, 42 views, 2,235 indexes.
- 4,556 LOB columns (longtext / longblob) — prevent INSTANT DDL on those tables.
- Mixed charsets: 24 tables utf8mb3, 1,033 tables utf8mb4.
- No partitioned tables, no stored procedures, no triggers.
- Module prefixes: cm(152), ln(126), fa(98), tr(77), pds(58), cus(46).

MySQL 8.4 DDL algorithms:

Algorithm	Duration	Locking
INSTANT	Milliseconds	Metadata-only — concurrent DML OK
INPLACE	Seconds–minutes	Online rebuild — concurrent DML OK
COPY	Minutes–hours	Full table rebuild — TABLE LOCKED

3.3 Solution

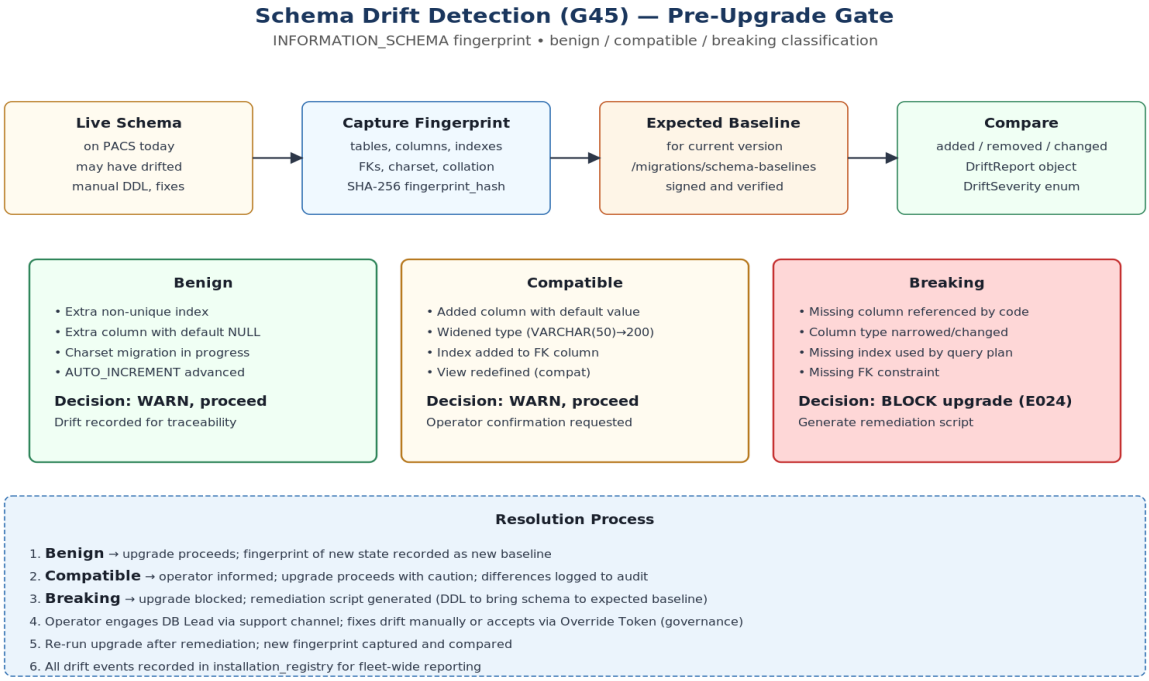

Figure: *Schema Drift Detection — Pre-Upgrade Gate*

Schema fingerprinting

- On install / upgrade: capture full INFORMATION_SCHEMA snapshot.
- Compute SHA-256 fingerprint hash.
- Before next upgrade: compare current vs expected baseline.
- Classify drift: Benign (extra indexes) → Compatible (additive columns) → Breaking (missing columns).
- Breaking drift → block upgrade + generate remediation script.

Migration ordering (FK-aware)

- Disable FK checks.
- Drop all 42 views.
- Infrastructure tables (Hangfire — charset conversion).
- Master tables (cm_* — FK parents).
- Customer tables (cus_* — FK parents for loans).
- Business tables (ln_*, fa_*, sca_*, trm_*).
- Trading / PDS / Asset tables.
- Audit / reporting tables.
- Re-enable FK checks.
- Recreate views from versioned definitions.
- Run pt-table-checksum on critical tables.

Migration script header (mandatory)

```
-- @migration: V025__add_updated_at_to_customer_tables.sql
-- @classification: INSTANT | INPLACE | COPY
-- @tables: cus_customerpersonaldetails, cus_customergeneral
-- @estimated_rows: 1500000
-- @estimated_duration: 30s
-- @requires_pt_osc: false
-- @phase: EXPAND | MIGRATE | CONTRACT
-- @rollback: V025__rollback.sql | NONE
-- @fk_dependencies: parent_table1, parent_table2
```

CI validates the header is present, parseable, and consistent with the script body.

3.4 Mitigation

Risk	Mitigation
Power-cut during migration	Checkpoint-per-script in schema_version_registry; resume from last committed
Manual DDL in the field	Schema fingerprint detects drift before upgrade; blocks if breaking
pt-online-schema-change fails	Old table untouched (shadow copy); retry or fall back to maintenance window
View breaks after column rename	Drop all views before migration; recreate from versioned definitions
FK constraint violation during migration	Disable FK checks during migration; re-enable + validate after

3.5 Pitfalls

Pitfall	Impact	Prevention
Fingerprint includes AUTO_INCREMENT values	False drift on every check	Exclude AUTO_INCREMENT from fingerprint (data, not structure)
LOB columns prevent INSTANT	Unexpected long migrations	DDL classifier identifies these at build time; CI validates
Mixed charset tables cause JOIN failures	Silent data corruption	One-time charset remediation in first upgrade (truncate Hangfire + ALTER)
pt-osc on table with triggers	pt-osc fails	ePACS has 0 triggers; if added, pt-osc v3.5+ handles them
Migration order wrong (child before parent)	FK violation	Topological sort from INFORMATION_SCHEMA.KEY_COLUMN_USAGE

4. Differential Backup and Data Sync

4.1 Technical query

How do we efficiently backup and sync a 1,057-table database that can grow to 100+ GB at DCCB hubs, over intermittent 4G connectivity?

4.2 Analysis

Three types of "delta" in the system:

- **Schema delta** — DDL changes between versions (DbUp migration scripts).
- **Data delta** — row-level changes for backup (Percona XtraBackup incremental, page-level).
- **Sync delta** — business data changes for NLDR (transactional outbox).

Backup challenge: full mysqldump of 100 GB = 15+ minutes + 100 GB disk space; daily full is impractical.

Sync challenge: only 9 tables have updated_at with ON UPDATE CURRENT_TIMESTAMP; 194 tables have DataEntryWorkingDate (INSERT only).

4.3 Solution

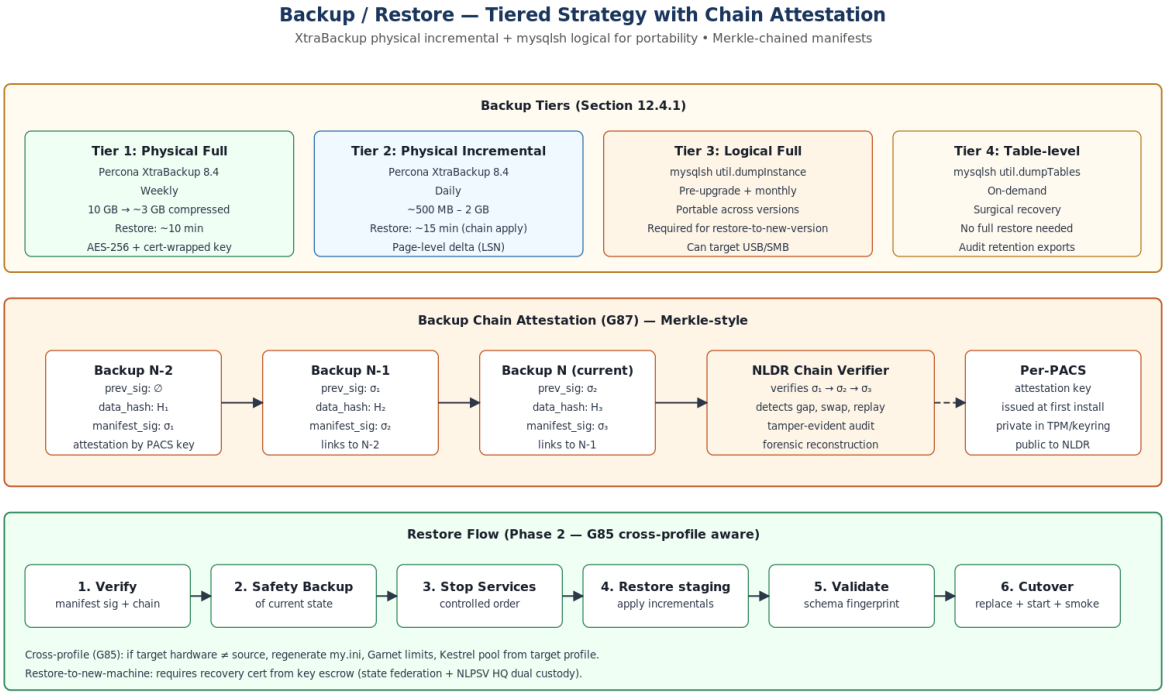

Figure: *Backup tiers and chain attestation*

Backup tiers

Tier	Tool	Type	When	Size (10 GB DB)
Physical Full	Percona XtraBackup 8.4	Full	Weekly	~10 GB

Tier	Tool	Type	When	Size (10 GB DB)
Physical Incremental	Percona XtraBackup 8.4	Incr (page-level)	Daily	~500 MB–2 GB
Logical Full	mysqlsh util.dumpInstance	Full logical	Pre-upgrade	~8 GB
Logical Table	mysqlsh util.dumpTables	Specific tables	On-demand	Varies

Sync delta tracking

- New transactions: captured via transactional outbox (existing pattern).
- Master data changes: add updated_at TIMESTAMP ON UPDATE CURRENT_TIMESTAMP — INSTANT DDL, zero downtime.
- Deletions: captured via outbox DELETE events (Section 2).
- Amendments: captured via outbox AMENDMENT events (Section 2).

4.4 Mitigation

- Percona XtraBackup for physical incremental — only changed InnoDB pages, genuinely differential.
- Content-hash deduplication for file sync — same file = sync once regardless of path.
- Priority-based sync drain on reconnect — financial → audit → master data → telemetry.
- Bandwidth-adaptive chunk sizing — 4G: 1 MB, 3G: 256 KB, 2G: 64 KB.

4.5 Pitfalls

Pitfall	Impact	Prevention
XtraBackup incremental chain breaks	Cannot restore	Weekly full resets chain; verify integrity daily
updated_at column backfilled with NULL	Cannot distinguish never-modified from migrated	NULL = never modified since migration; treat as original
Large LOB columns inflate incrementals	Daily incremental larger	LOB changes are rare; monitor incremental size
Sync outbox grows unbounded	Disk exhaustion	Soft 500K / hard 2M ceilings (G66); financial events never deferred

5. File / Attachment Sync

5.1 Technical query

How do we sync member photos, field-verification uploads, and generated reports between PACS and NLDR?

- ~2 MB/day per PACS (photos + uploads).
- Reports 1–10 MB each (on-demand).
- Bidirectional (PACS → NLDR for uploads; NLDR → PACS for policies).
- Intermittent 4G connectivity.
- Enable / disable feature flag.

5.2 Analysis

File naming convention: {state_id}/{dccb_id}/{branch_id}/{pacs_id}/{type}/{filename}

Storage: D:\ePACSData\attachments\ with hierarchical structure.

Transport options: SFTP (resume support, large files), HTTPS multipart (firewall-friendly).

5.3 Solution

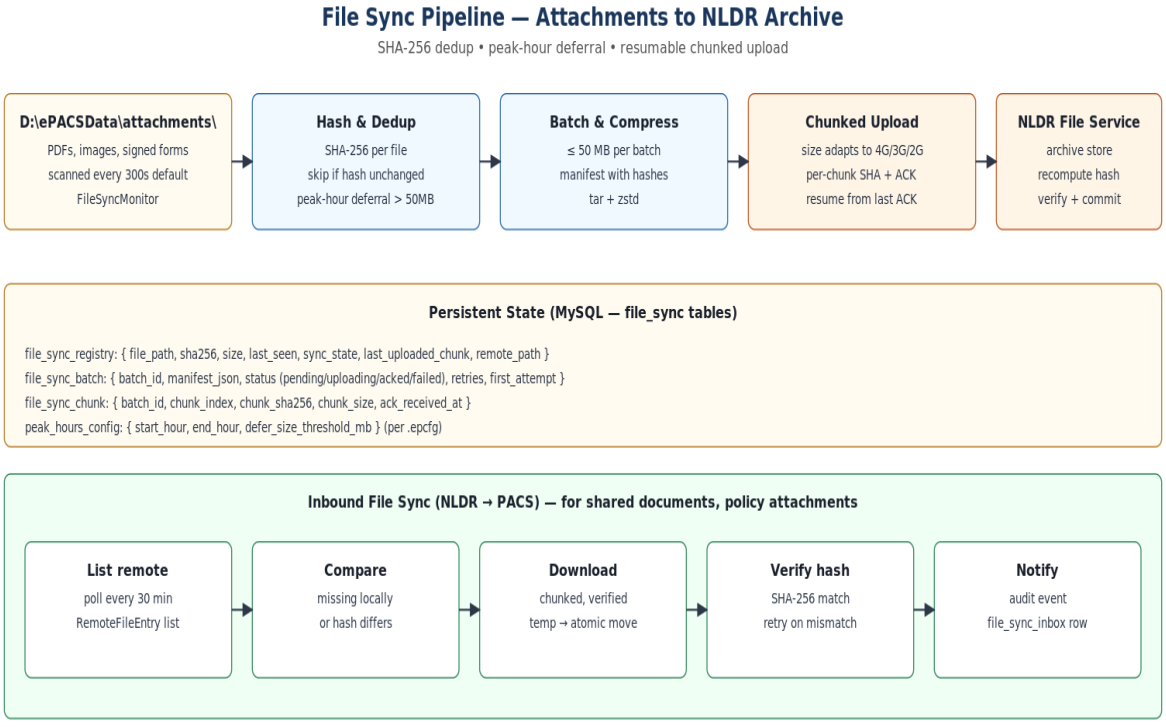

Figure: *File sync pipeline — content-hash dedup, peak-hour deferral*

Content-hash-based deduplication

- Compute SHA-256 of each file.

- Store in file_sync_registry MySQL table.
- If hash unchanged since last sync → skip (no re-upload, even if file renamed).
- Dual transport: HTTPS primary, SFTP fallback (configurable).

Enable / disable

- FileSync.Enabled = true/false in appsettings.json.
- Controllable via ePACS application UI at runtime.
- When disabled: files accumulate locally; registry tracks them.
- When enabled: backlog drains automatically (small files first).

5.4 Mitigation

- Batch-size limit per sync cycle (default 50 MB).
- Priority: small files first (photos ~200 KB), then reports (1–10 MB).
- Peak-hours awareness: skip large-file hashing during business hours.
- Resume: per-chunk ACK for HTTPS; SFTP native resume.

5.5 Pitfalls

Pitfall	Impact	Prevention
Large PDF blocks small-photo sync	Photos delayed	Priority queue: small files first
SFTP key compromised	Unauthorized access	Key rotation via signed config update; per-PACS key
File deleted locally after sync	NLDR has file PACS doesn't	File deletion events in sync (mirror DB pattern)
Same file uploaded with different name	Wasted bandwidth	Content-hash dedup prevents re-upload
USB-copied files have future timestamps	Sync logic confused	Use content hash, not timestamp, as the delta key

6. PACS Heartbeat and Online / Offline Detection

6.1 Technical query

How does NLDR / CoopsIndia Dashboard know if a PACS is online or offline? How does a PACS proactively announce its connectivity status?

6.2 Analysis

NLDR has no way to ping a PACS (PACS is behind NAT / firewall, no inbound). The PACS must proactively announce.

6.3 Solution

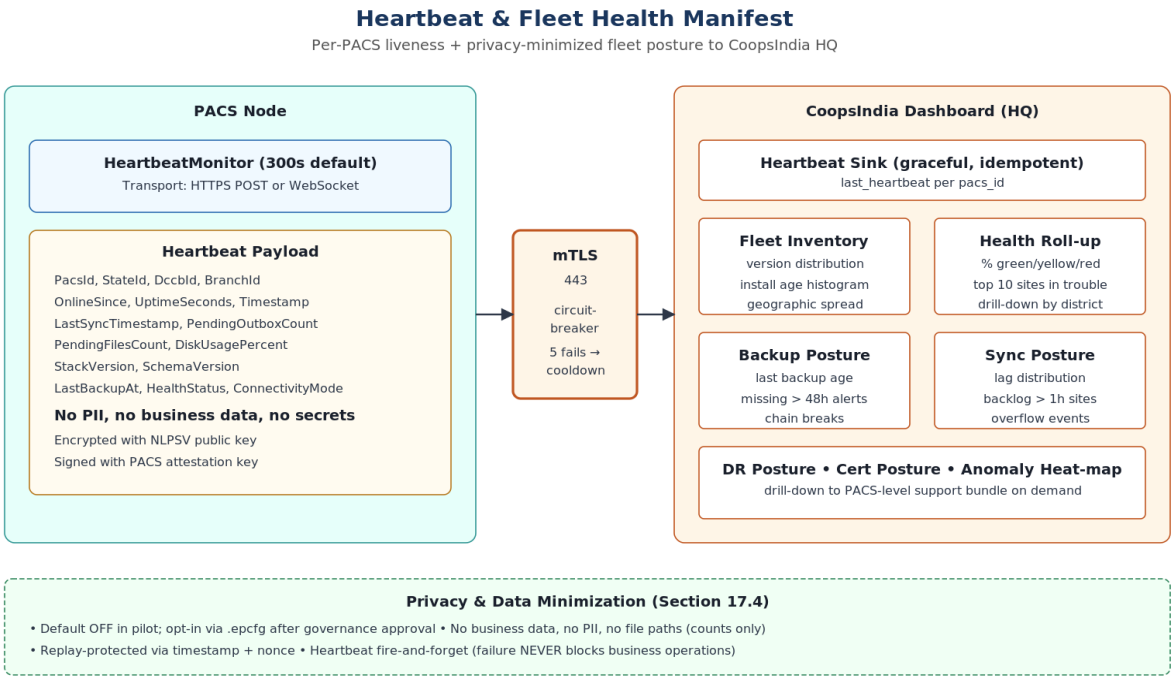

Figure: *Heartbeat from PACS to CoopsIndia Dashboard*

Behavior

- On connectivity detected → send immediate heartbeat.
- While online → send every 5 minutes (configurable).
- On connectivity lost → stop sending.
- Dashboard marks PACS offline after 2× interval (10 min) with no signal.

Dual protocol

- HTTPS POST to /api/v1.0/pacsStatus (stateless, firewall-friendly).
- WebSocket to /ws/v1.0/pacsStatus (real-time, persistent).
- Selected via configuration.

Heartbeat payload (~< 1 KB)

```
{
 "pacs_id": "AP-XYZ-0001",
 "state_id": "AP",
 "dccb_id": "XYZ",
 "branch_id": "001",
 "online_since": "2026-05-04T10:00:00Z",
 "last_sync_timestamp": "2026-05-04T09:45:00Z",
 "pending_outbox_count": 42,
 "pending_files_count": 7,
 "disk_usage_percent": 35,
 "stack_version": "3.2.1",
 "schema_version": 25,
 "last_backup_at": "2026-05-04T02:00:00Z",
 "health_status": "Healthy",
 "connectivity_mode": "4G",
 "uptime_seconds": 3600
}
```

6.4 Mitigation

- Fire-and-forget: heartbeat failure NEVER blocks business operations.
- Circuit breaker: after 5 consecutive failures, stop for cooldown.
- Bandwidth-minimal: payload < 1 KB; negligible even on 2G.

6.5 Pitfalls

Pitfall	Impact	Prevention
Heartbeat endpoint down	Dashboard shows all PACS offline	Distinguish endpoint-down from PACS-offline at dashboard
Clock drift affects online_since	Misleading uptime data	Use uptime_seconds (relative) for duration
Heartbeat floods on reconnect	Dashboard overwhelmed	Rate-limit: max 1 heartbeat per interval regardless of events
WebSocket connection leak	Memory exhaustion	Reconnect with exponential backoff; max 1 connection per PACS

7. Rural Resilience and Power-Cut Recovery

7.1 Technical query

How do we ensure the installer and all services survive hard power loss at any point during any operation?

7.2 Analysis

Environment reality:

- UPS not guaranteed at all PACS sites.
- Power cuts are a daily occurrence in rural India.
- 8 GB RAM + SSD hardware (no spinning disks).
- 35–45 °C operating temperatures.

7.3 Solution

> Every operation is resumable from a checkpoint. No operation is allowed to leave the system in an unrecoverable state. State.json is the universal recovery anchor.

Recovery matrix

Operation	Recovery behavior
Fresh install — payload extract	Resume from last extracted payload (staging-manifest.json)
Fresh install — DB init	Drop and re-initialize (no data yet)
Upgrade — binary staging	Junction still points to old version → old version starts normally
Upgrade — DB migration	Resume from last committed script (schema_version_registry checkpoint)
Upgrade — junction flip	Junction not yet flipped → old version still active
Backup in progress	Incomplete backup discarded (missing manifest sig); previous valid backup retained
Restore in progress	Pre-restore safety backup exists → revert to safety backup
Business operation	InnoDB crash recovery → MySQL restarts → services restart via Windows recovery
Sync upload	Resume from last ACK'd chunk (checkpoint in MySQL)
Audit write	Transactional sink → InnoDB guarantees; deferred journal as fallback

MySQL hardening

```
innodb_flush_log_at_trx_commit = 1     # fsync redo log on every commit
sync_binlog                    = 1     # fsync binlog on every commit
innodb_doublewrite             = ON    # protect against torn pages
innodb_checksum_algorithm      = crc32 # detect corruption
```

Installer state checkpoint

- Write state.json with fsync on every phase transition.
- Use write-then-rename pattern (atomic on NTFS).

- On restart: read state.json → resume from recorded phase.

7.4 Mitigation

- Atomic file operations — write-then-rename for all config/state files.
- Transaction boundaries — every business operation is a single MySQL transaction.
- Service recovery actions — Windows manager restarts crashed services (60s/120s/300s).
- Installer Agent — detects incomplete state on boot → enters RECOVERY mode.

7.5 Pitfalls

Pitfall	Impact	Prevention
state.json itself is corrupt	Cannot determine resume point	Write-then-rename makes torn writes impossible on NTFS
MySQL redo log corrupt after power-cut	DB won't start	innodb_doublewrite=ON; InnoDB crash recovery handles the rest
Kafka log corrupt	Events lost	flush.messages=1; but Kafka is NOT the durable anchor — MySQL outbox is
SSD wear from frequent fsync	Reduced lifespan	Modern SSDs handle millions of cycles; RAM buffers most writes
Thermal throttling during long migration	Migration takes hours	Monitor temperature via WMI; pause at 85°C; reduce parallelism

8. Observability and Error Handling

8.1 Technical query

How do we ensure consistent, correlated, PII-safe logging across all services, and how do we handle errors usefully for both operators and support engineers?

8.2 Analysis

Existing platform utilities:

- Intellect.Erp.Observability (10 NuGet packages): structured logging, correlation, redaction, audit hooks.
- Intellect.Erp.ErrorHandling: typed exceptions, YAML error catalogs.
- Intellect.Erp.Traceability: compliance-grade audit (11 tables, geo-tag, anomaly rules).

8.3 Solution

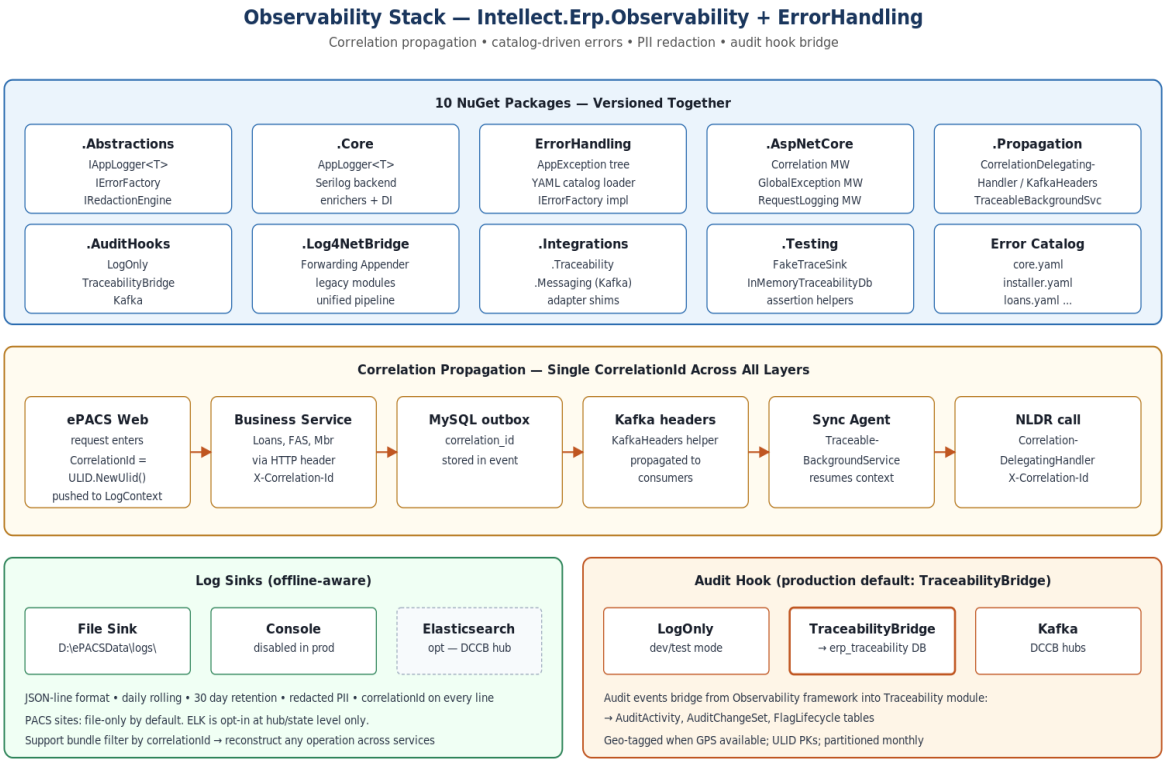

Figure: *Observability stack — correlation, redaction, audit hooks*

Structured logging schema (v1)

Every log entry carries:

```
@timestamp, level, messageTemplate, message,
correlationId, app, env, machine, module,
tenantId, stateCode, pacsId, feature, operation,
checkpoint, <message-template parameters>
```

Error handling

- YAML error catalog: ERP-INST-{CATEGORY}-{NUMBER} (e.g., ERP-INST-PRE-0004).
- Typed exceptions: PrecheckException, InstallException, MigrationException, …
- Operator sees: plain-language userMessage.
- Support bundle contains: supportMessage with technical detail + correlationId.

Audit trail

- Hash-chained audit log for critical operations (tamper-evident).
- Traceability module integration via TraceabilityBridgeAuditHook.
- Geo-tagged with fallback chain: GPS → Cell Tower → Site Config → Unavailable.

8.4 Mitigation

- Support bundle — one-click generation, encrypted ZIP, PII redacted, correlation-filtered.
- Log rotation — 30-day app, 90-day audit, 7-day MySQL (configurable).
- Disk protection — max 10% of data partition or 50 GB for logs (whichever smaller).

8.5 Pitfalls

Pitfall	Impact	Prevention
PII leaks via exception messages	Compliance violation	IRedactionEngine on all log output; regex for Aadhaar/mobile/account
CorrelationId lost across Kafka boundary	Cannot trace end-to-end	KafkaHeaders helper propagates correlationId in headers
Error catalog code not found	Generic error to operator	Fallback ERP-CORE-SYS-0001 with original message in support log
Log files fill disk	Services crash	Installer Agent enforces rotation; critical threshold blocks writes
Audit chain broken (tampered)	Cannot prove integrity	Chain verification on every Installer Agent health check; alert on break

9. Configuration Architecture

9.1 Technical query

How do we ensure zero hardcoding while maintaining sensible defaults and supporting site-specific customization across hundreds of PACS?

9.2 Solution

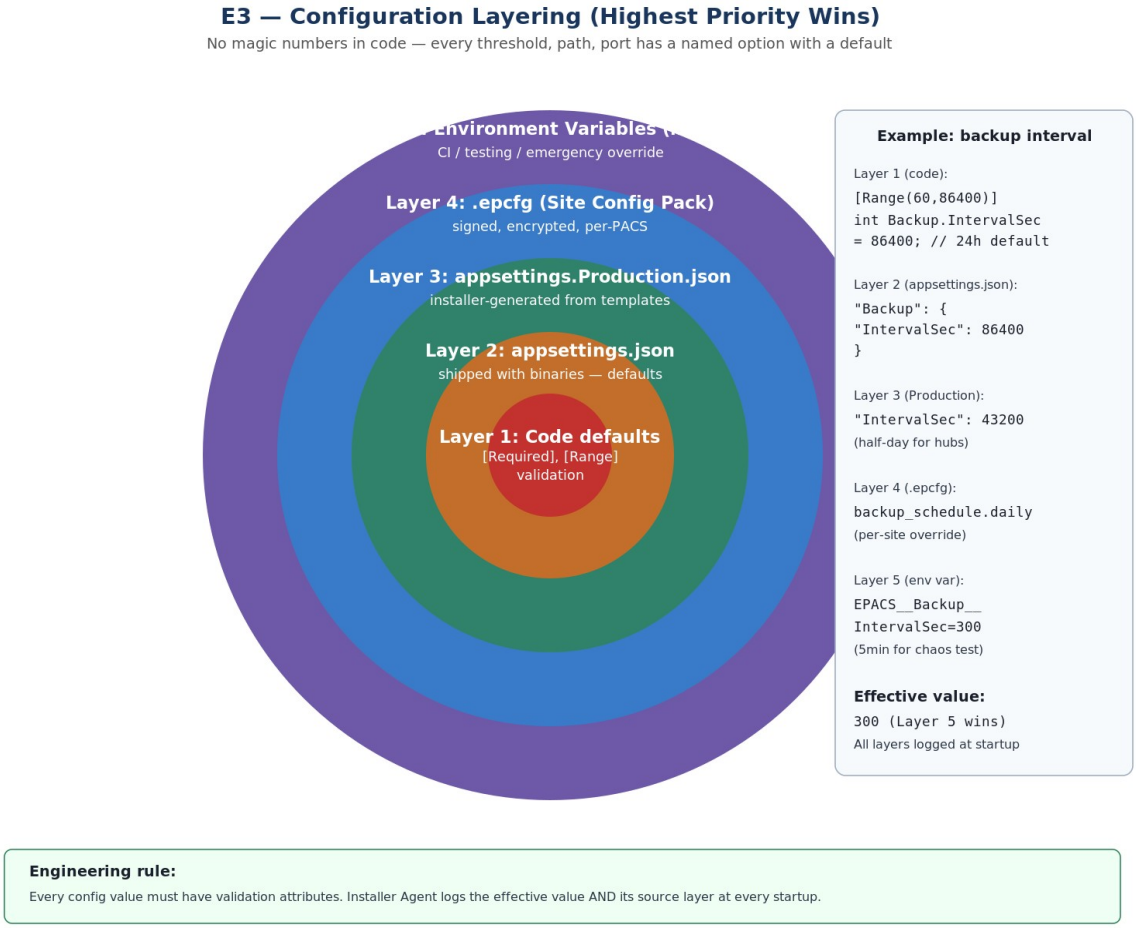

Figure: *Configuration layering — five layers, highest priority wins*

Configuration hierarchy (highest priority wins)

- Layer 1 — Code defaults: strongly-typed Options classes with [Required] and [Range] validation.
- Layer 2 — appsettings.json: shipped with binaries, defaults.
- Layer 3 — appsettings.Production.json: installer-generated from templates.
- Layer 4 — .epcfg (Site Config Pack): site-specific values, signed and encrypted.
- Layer 5 — Environment variables: runtime overrides for CI / testing / emergencies.

Key principle: every threshold, path, port, interval, and retry count is a named configuration value with a

sensible default. No magic numbers in code. The Installer Agent logs the effective value AND its source layer at every startup.

9.3 Pitfalls

Pitfall	Impact	Prevention
Operator edits appsettings.json manually	Drift; upgrade overwrites changes	Drift detection (hourly); repair mode regenerates from templates
.epcfg signature invalid after manual edit	Installer refuses to use it	Clear error: "Site configuration pack signature is invalid."
Env var overrides forgotten	Unexpected behavior in prod	Installer Agent logs all active config sources at startup
Default inappropriate for large PACS	Performance issues	Hardware-profile detection (small / medium / large) with different defaults

10. Critical Rules for Development Team (Source Guide)

These ten rules are reproduced verbatim from docs/engineering-guide.md and remain the canonical short list. The remaining sections of this document show how each rule is enforced and what to do if you must work around it.

1. Never hardcode paths, ports, thresholds, or credentials. Everything from configuration.
2. Every DELETE must write to sync_outbox in the same transaction (before-state captured).
3. Every amendment must include reason + approver (configurable enforcement).
4. AUTO_INCREMENT seed = pacsid × 10^{10} — never manually set AUTO_INCREMENT.
5. Every state transition writes a checkpoint (fsync'd, write-then-rename).
6. PII never appears in logs — use [Sensitive], [DoNotLog], [Mask] attributes.
7. Heartbeat / sync failures never block business operations — fire-and-forget with circuit breaker.
8. Schema migrations classified by DDL algorithm — COPY on > 1M rows uses pt-online-schema-change.
9. Backup before every destructive operation — mandatory, not optional.
10. Test with power-cut simulation — every long operation must survive hard shutdown.

11. Coding Standards & Conventions

11.1 Language and platform

- .NET 8 LTS, C# 12. No language-feature crystal-balling — use only features GA'd in 8.0.
- Self-contained publish (win-x64) — eliminates runtime drift across field sites.
- Nullable reference types: enabled (#nullable enable in csproj). Treat warnings as errors.
- LangVersion: latest stable; PublishTrimmed: false (false-positive trimming risk on reflection).

11.2 Async and threading

- Public async APIs return Task or ValueTask; return Task in interfaces; ValueTask only when measured benefit.
- ConfigureAwait(false) is NOT used in installer code — installer is single-threaded; prevents subtle bugs.
- ConfigureAwait(false) IS used in business services and Sync.Agent (multi-threaded).
- Never call .Result or .Wait() — sync-over-async deadlocks under SynchronizationContext.
- CancellationToken propagated through every async method signature.

11.3 Exceptions

- Throw typed exceptions from Intellect.Erp.ErrorHandling. Never throw bare Exception or InvalidOperationException.
- Every exception carries a catalog code (ERP-INST-...). The IErrorFactory is the only place that constructs them.
- Catch only what you can handle. Re-throw with throw; (preserves stack), not throw ex;.
- Never catch Exception except in top-level workers (TraceableBackgroundService does this for us).

11.4 Logging

- Use IAppLogger<T>, never raw Microsoft.Extensions.Logging.
- Log message templates use named placeholders ({UpgradeId}, not {0}).
- Never concatenate user input into log messages — always parameterize.
- Log levels: Debug (dev only), Information (state transitions), Warning (degraded), Error (recoverable failures), Critical (operator-attention).
- PII attributes ([Sensitive], [Mask], [DoNotLog]) on DTOs — Roslyn analyzer enforces.

11.5 Dependency injection

- Constructor injection only. No service locator. No static state.
- Lifetime: Singleton for stateless utilities, Scoped for per-request, Transient for ephemeral.
- Avoid IServiceProvider in business code — only in composition root.
- Options pattern (IOptions<T>, IOptionsSnapshot<T>) for configuration; never IConfiguration directly.

11.6 Database access

- EF Core for business CRUD; ADO.NET (Dapper) for installer / migration / pt-osc orchestration.
- Always parameterized — never string-concatenate SQL.
- AsNoTracking() for SELECTs that don't update (huge perf win on large reads).
- Explicit columns — never SELECT *. Avoids surprises after schema changes.
- Transactions explicit (BeginTransactionAsync), not relying on default behavior.
- MySQL advisory lock GET_LOCK('epacs_migration', 0) at top of migration runner.

11.7 Naming

Element	Convention
Class / record / struct	PascalCase (BackupEngine, SyncEvent)
Method / property	PascalCase (CreateBackupAsync, IsHealthy)
Local / parameter	camelCase (backupId, ct)
Constant	PascalCase (DefaultRangeSize), not SCREAMING_CASE
Interface	I-prefix (ISyncTransport)
Async method	Async-suffix (SendBatchAsync) — required by FxCop
Boolean	Is/Has/Can/Should-prefix (IsHealthy, HasOutbox)
Database column	PascalCase OR existing convention (don't migrate column casing)
Configuration key	MixedCase, dot-separated (Backup.IntervalSec)
Catalog code	ERP-{MOD}-{CAT}-{NNNN} (ERP-INST-PRE-0004)

11.8 File and project layout

- One public type per file. File name matches type name.
- Folders by feature, not by layer. /Sync/Outbox/OutboxRelay.cs, not /Services/OutboxRelay.cs.
- Tests mirror project layout — tests/Sync.Agent.Tests/Outbox/OutboxRelayTests.cs.

11.9 Comments and documentation

- Public APIs: XML doc-comments mandatory. Generate API docs in CI.
- Internal: comment why, not what. Code says what; comment explains intent or trade-off.
- // TODO: forbidden in main. Use issue tracker instead.
- Banned phrases in comments: "obviously", "trivially", "simply" — usually they're not.

12. Branch Strategy and CI/CD Pipeline

12.1 Branch model — trunk-based with short-lived feature branches

- main — protected; always release-able; signed commits required.
- feature/<ticket-id>-<short-desc> — one branch per PR; max 5 days alive.
- hotfix/<incident-id> — emergency fix for production; cherry-picked back to main.
- release/<version> — stabilization branch for major releases only; bug-fix-only.

12.2 Commit and PR conventions

- Conventional commits: feat:, fix:, refactor:, docs:, test:, chore:, perf:.
- Commit messages: imperative mood, < 72 chars subject; body explains why.
- Signed commits required (GPG or SSH key registered with NLPSV identity).
- PR template enforces: linked ticket, scope, test evidence, security-review tag.

12.3 PR review

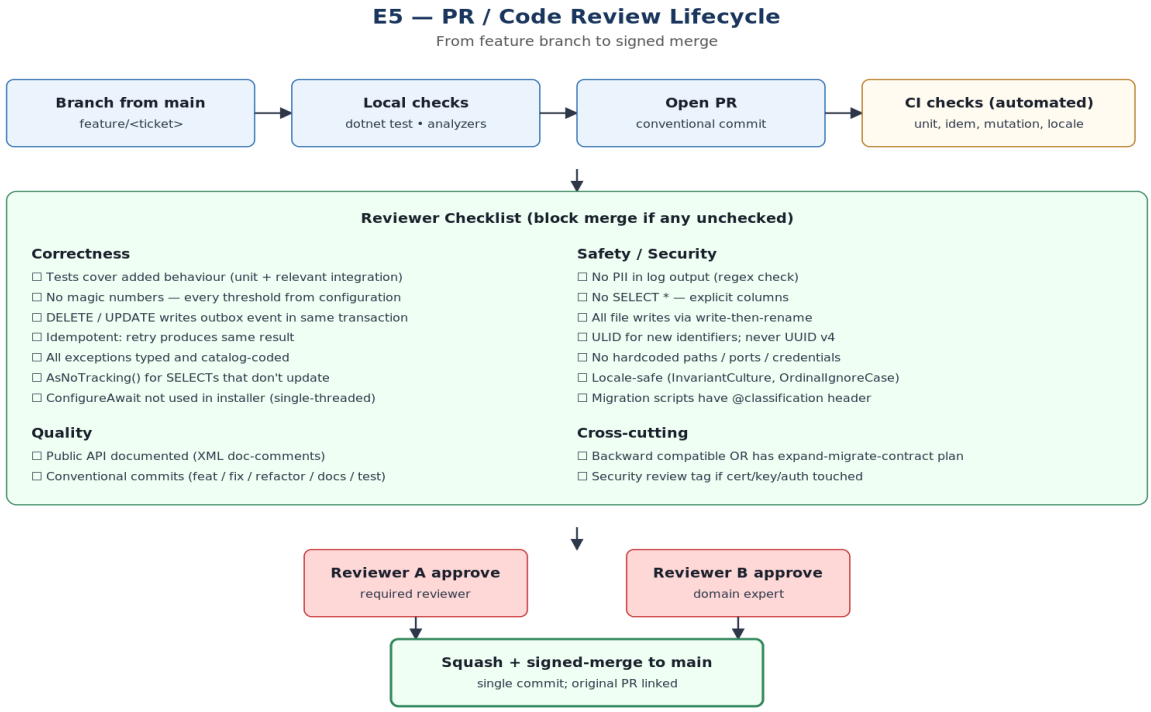

Figure: *PR / Code Review Lifecycle*

12.4 CI/CD pipeline

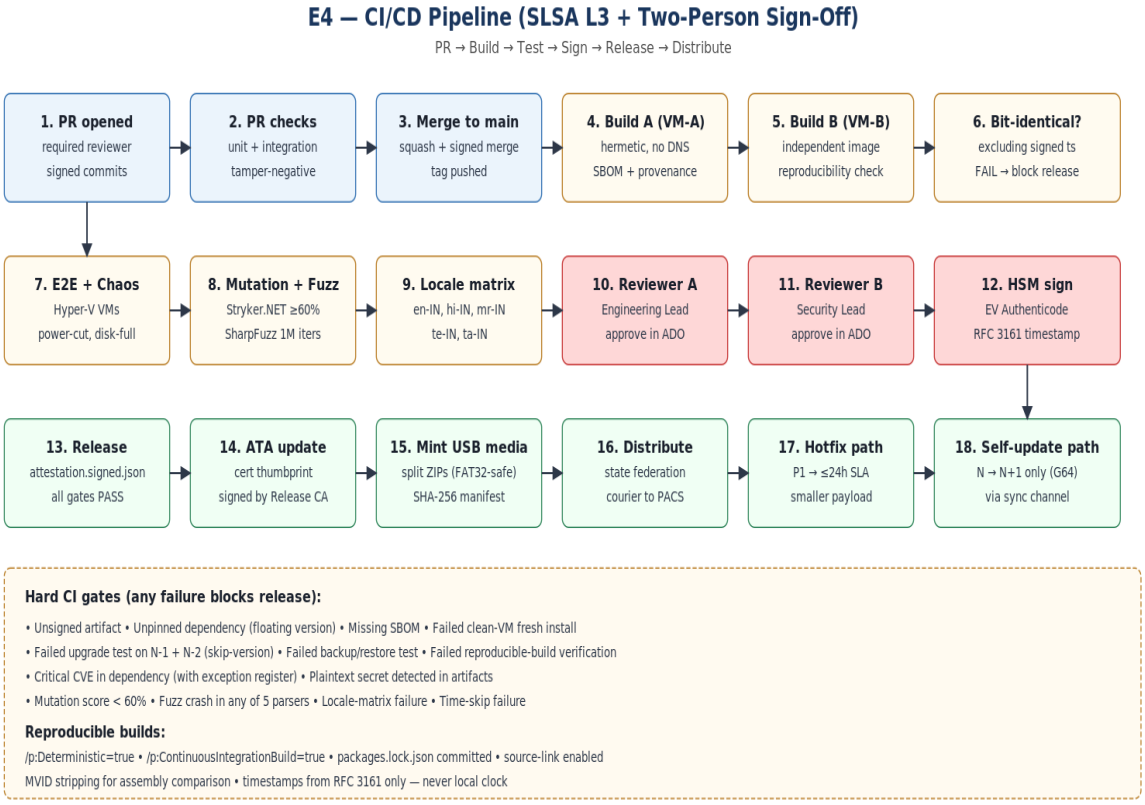

Figure: *CI/CD pipeline — SLSA L3 with two-person sign-off*

12.5 Release process

Release type	Process
Patch (3.2.0 → 3.2.1)	Bug fixes only; binaries-only payload (~200 MB); no schema migration; weekly cadence
Minor (3.2 → 3.3)	New features; may include INSTANT/INPLACE migrations; quarterly cadence
Major (3.x → 4.0)	Breaking changes; full payload; expand-migrate-contract phases; yearly cadence
Hotfix	P1 only; ≤ 24h SLA; smaller signed package; replaces only changed binaries; promoted to next full release
Self-update (G64)	Installer-only patches; N → N+1 only; via signed sync-channel manifest

13. Local Development Environment

13.1 Prerequisites

- Windows 11 or macOS (with Hyper-V / Parallels for VM-fidelity testing).
- 16 GB RAM minimum, 100 GB SSD free.
- .NET 8 SDK (8.0.x — pinned in global.json).
- Docker Desktop OR Hyper-V.
- Git with signed-commits configured (gpg --gen-key OR SSH key registered).
- MySQL Workbench or DataGrip for DB inspection.
- Visual Studio 2022 17.10+ OR Rider 2024.1+ OR VS Code with C# extension.

13.2 Setup

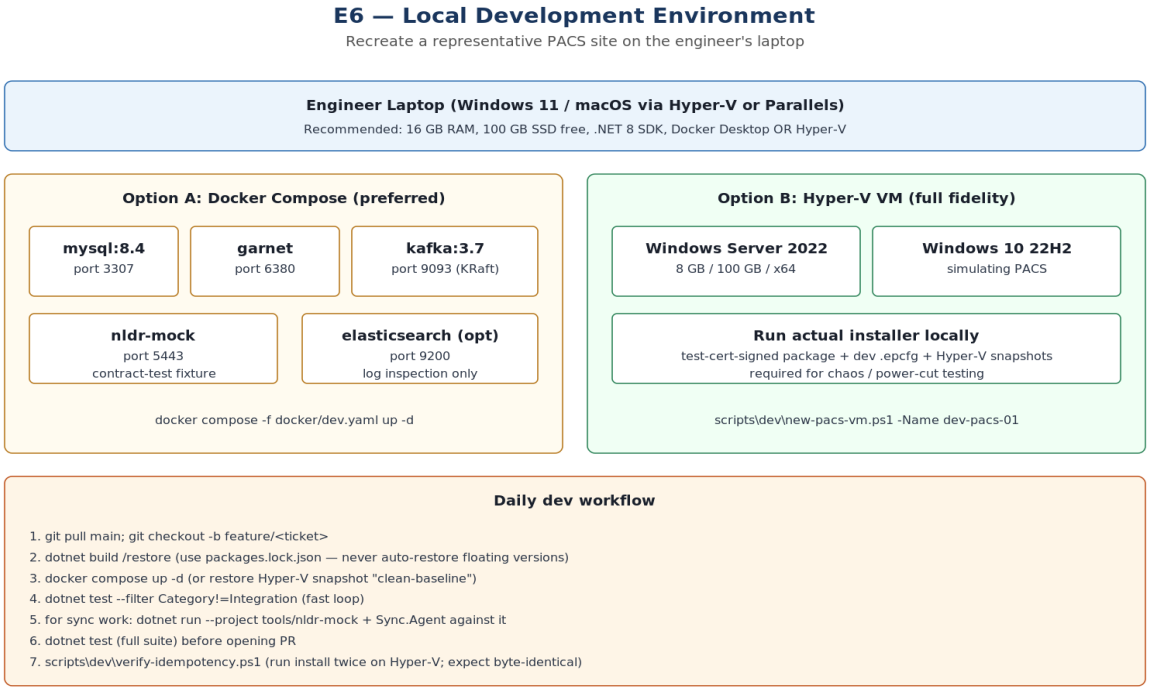

Figure: *Local development environment — Docker Compose + Hyper-V VMs*

Day-1 setup

```
# 1. Clone
git clone https://nlpsv.dev/_git/l3_installer
cd l3_installer

# 2. Restore + build
dotnet restore   # uses packages.lock.json (committed)
dotnet build

# 3. Spin up local stack
docker compose -f docker/dev.yaml up -d
# Or for full-fidelity:
```

```
scripts/dev/new-pacs-vm.ps1 -Name dev-pacs-01

# 4. Apply schema
dotnet run --project src/Installer.Core.SchemaMigration -- --target localhost:3307

# 5. Run tests
dotnet test --filter Category!=Integration   # fast loop
dotnet test                                  # full

# 6. Run installer locally
dotnet run --project src/Installer.CLI -- --config samples/dev.epcfg --quiet
```

13.3 Daily workflow

- git pull main; git checkout -b feature/<ticket>
- dotnet build / restore (use packages.lock.json — never auto-restore floating versions).
- docker compose up -d (or restore Hyper-V snapshot "clean-baseline").
- dotnet test --filter Category!=Integration (fast loop).
- For sync work: dotnet run --project tools/nldr-mock + Sync.Agent against it.
- dotnet test (full suite) before opening PR.
- scripts\dev\verify-idempotency.ps1 (run install twice on Hyper-V; expect byte-identical).

13.4 Useful tools

Tool	Purpose
scripts/dev/reset-db.ps1	Drop + recreate local MySQL with seed data
scripts/dev/new-pacs-vm.ps1	Provision a Hyper-V VM mimicking a PACS site
scripts/dev/snapshot-vm.ps1	Take a Hyper-V snapshot before destructive testing
scripts/dev/verify-idempotency.ps1	Run install twice, diff system state
scripts/dev/inject-power-cut.ps1	Force-stop VM mid-operation for chaos testing
tools/nldr-mock	In-memory NLDR fixture with contract-test endpoints
tools/log-tail	Tail-and-filter local JSON logs by correlationId
tools/epacs-ping (release build)	Operator-style end-to-end health check

14. Code Review Checklist

Reviewers MUST run this checklist on every PR. Failing any item is a blocking comment.

14.1 Correctness

- Tests cover added behaviour (unit + relevant integration).
- No magic numbers — every threshold from configuration.
- DELETE / UPDATE writes outbox event in the same transaction (Section 2).
- Idempotent — retry produces same result (Section 7).
- All exceptions typed and catalog-coded (Section 8).
- AsNoTracking() for SELECTs that don't update.
- CancellationToken propagated through every async chain.
- Locale-safe (InvariantCulture, OrdinalIgnoreCase, NFC normalization).

14.2 Safety / security

- No PII in log output (regex check + golden-list test).
- No SELECT * — explicit columns.
- All file writes via write-then-rename atomic pattern.
- ULID for new identifiers; never UUID v4 except for transient (correlationId).
- No hardcoded paths / ports / credentials.
- No secrets in commit history (git secret-scan in CI).
- Migration scripts have @classification, @phase, @rollback headers (Section 3).

14.3 Quality

- Public API documented (XML doc-comments).
- Conventional commits (feat / fix / refactor / docs / test / chore / perf).
- PR description links to ticket; explains why; lists testing evidence.
- Diff size reasonable (< 600 lines net change preferred; > 1500 needs explanation).

14.4 Cross-cutting

- Backward compatible OR has expand-migrate-contract plan.
- Security review tag if cert / key / auth / sync code touched.
- Performance review tag if hot-path code touched (logged or measured).
- Documentation updated (this guide, SAD, ADRs) if architectural impact.

14.5 Reviewer reminders

- Be kind, be specific, suggest fixes — not just "this is wrong".
- Use "nit:" prefix for non-blocking style comments.
- Approve with comments OR Request changes — never silent approve when you have concerns.

- Reviewer A and Reviewer B should be different humans; the bot "required reviewer" rule enforces this.

15. Debugging Playbooks

15.1 Symptom-to-playbook index

Symptom	Playbook
Installer fails with E022 (SBOM mismatch)	P1: tampered media or build-pipeline issue
Installer hangs at MIGRATE phase	P2: stuck migration
Sync stuck in OPEN state for hours	P3: connectivity / TLS handshake debug
sync_outbox growing past 500K events	P4: outbox backpressure investigation
Operator reports "app slow" after upgrade	P5: post-upgrade performance regression
MySQL won't start after power-cut	P6: InnoDB recovery
Garnet AOF crash loop	P7: cache corruption
Heartbeat shows "Healthy" but operator reports issues	P8: false-positive health check
NLDR rejects sync events with hash mismatch	P9: wire-format / canonicalization issue

15.2 P1 — SBOM mismatch (E022)

- Compare release-sbom.json hash to installer-recomputed hash from logs.
- Identify which file mismatches; check Authenticode signature on the file.
- If signature valid but SBOM mismatch → suspected build-pipeline issue; escalate to Security Lead.
- If signature invalid → tampered media; quarantine; request fresh USB.
- Never bypass with "force install" — use Compromise Runbook (SAD Appendix M) if confirmed.

15.3 P2 — Stuck migration

- Check schema_version_registry: SELECT * ORDER BY applied_at DESC LIMIT 5.
- Identify last "in-progress" script. Read its @classification header.
- If COPY-algorithm on > 1M rows: check pt-osc log file (D:\ePACSData\logs\pt-osc\).
- If pt-osc shadow table exists: SHOW PROCESSLIST to confirm copy in progress.
- If process truly stuck: KILL <id>; pt-osc cleans up shadow; restart migration runner.
- Never manually run ALTER on the actual table while migration is in progress.

15.4 P3 — Sync OPEN circuit

- Check Sync.Agent log for last successful probe time.
- From PACS host: tracert <NLDR-FQDN>; verify DNS resolution.
- openssl s_client -connect <NLDR>:443 -showcerts — verify TLS handshake.
- If cert chain broken: root-CA bundle stale (G71); apply latest root-CA snapshot.
- If 4G connectivity OK but TLS fails: check NLDR cert expiry; check ATA refresh status.

- Probe manually: curl -I https://<NLDR>/health; expect 200.

15.5 P4 — Outbox backpressure

- SELECT COUNT(*) FROM sync_outbox WHERE status='PENDING'; — confirm threshold breach.
- SELECT change_type, COUNT(*) FROM sync_outbox GROUP BY change_type; — identify category.
- If financial events disproportionate: investigate why drain isn't draining (P3).
- If telemetry events dominant: confirm overflow table is receiving them (sync_outbox_overflow).
- Operator-visible "days to ceiling" gauge in dashboard — share with field SRE.

15.6 P5 — Post-upgrade performance regression

- Compare schema fingerprint: pre-upgrade vs post-upgrade; expect same or known-additive.
- EXPLAIN on slow queries; check for missing indexes after migration.
- Check buffer pool size in my.ini; if hardware profile mismatched (G85) → regenerate config.
- If specific table slow: pt-table-usage to identify hot rows; consider partition or index.
- Last-resort: rollback junction to previous version; root-cause in dev environment.

15.7 P6 — InnoDB recovery

- Check MySQL error log: D:\ePACSData\mysql\logs\error.log.
- InnoDB performs crash recovery automatically on startup; wait up to 10 minutes.
- If recovery fails: innodb_force_recovery = 1 (read-only); export essential data; full restore from backup.
- Never delete ib_logfile* — that destroys the redo log.

15.8 P7 — Garnet AOF crash

- Garnet log: D:\ePACSData\cache\garnet.log.
- If AOF corrupt: rename appendonly.aof → appendonly.aof.corrupt; restart Garnet.
- Cache will refill from MySQL — no data loss (cache is non-authoritative).
- Audit the corruption event in Traceability; investigate root cause.

15.9 P9 — Hash mismatch on sync

- Hash is computed over canonicalized JSON (RFC 8785 JCS).
- Verify both ends use identical canonicalization; common bug: different float precision.
- Check decimal precision: schema migration may have widened/narrowed; G89 monitor flags this.
- Compare wire payload byte-for-byte between PACS sender and NLDR receiver via tcpdump (with TLS keylog).

16. Performance Budget

16.1 Per-operation targets

Operation	Small PACS (8 GB)	DCCB Hub (32 GB)	Notes
Fresh install	< 15 min	< 12 min	End-to-end including services start
Patch upgrade	< 10 min	< 8 min	Binaries-only payload
Minor upgrade + 10 GB DB	< 30 min	< 20 min	Including migration
Major upgrade + 10 GB DB	< 45 min	< 30 min	Side-by-side staging
Major upgrade + 100 GB DB	< 4 hours	< 2 hours	XtraBackup + pt-osc
Backup 10 GB DB	< 10 min	< 5 min	Physical incremental
Backup 100 GB DB	< 90 min	< 30 min	Weekly full
Restore 10 GB DB	< 15 min	< 10 min	Same-machine
Restore 100 GB DB	< 4 hours	< 1.5 hours	Includes prepare
Hotfix apply	< 5 min	< 5 min	Affected services only
Sync drain (10K events)	≥ 1,000 / min	≥ 3,000 / min	Steady state
Health probe P95 latency	< 500 ms	< 250 ms	/health/ready cached 10s
epacs-ping diagnostic	< 30 sec	< 15 sec	Operator one-shot

16.2 Memory budget per service

Service	Steady-state RAM (Small PACS)	Notes
MySQL	2 GB (innodb_buffer_pool)	Sized to 25 % of RAM
Garnet	512 MB	AOF + snapshot
Kafka	1 GB	JVM + page cache
ePACSWeb (Kestrel)	256 MB	Per worker
Business services (each)	200 MB	Loans, FAS, etc.
Sync.Agent	150 MB	Includes Polly, batch buffer
Installer.Agent	100 MB	Always-on watchdog
Total ePACS	~ 5 GB	Leaves 3 GB for OS + headroom

16.3 Hot paths — extra scrutiny

- Outbox.Relay drain loop: every batch shaped, allocations tracked. Use ArrayPool<byte> for serialization buffers.
- InboxProcessor: parallelism capped (Environment.ProcessorCount/2); avoid contending on MySQL writes.

- FileSyncMonitor: hash compute is the bottleneck — buffered IO, channel-based pipeline.
- Kestrel /health: cached by middleware (10 s for /ready, 1 s for /live) — never touches DB on hot path.

16.4 Tail-latency rules

- P50, P95, P99 measured for every endpoint, not just averages.
- P99 target = P50 × 5. If your P99 exceeds this ratio, hunt down the long tail.
- Common tail causes on rural hardware: GC pause, AV scan, log rotation, SSD GC. Each requires a different fix.

17. Security Review Checklist (PR Triggers)

Any PR matching the criteria below requires Security Lead approval (Reviewer B = Security Lead).

17.1 Triggers

- Touches anything under /SharedKernel/Security/.
- Touches certificate, key, or signing logic (ICodeSigner, IAclEngine, ISecretStore).
- Adds or modifies Override Token issuance / verification.
- Adds or modifies ATA, root-CA bundle, or SBOM verification.
- Modifies encryption (AES, key wrapping) or hashing in audit / backup chains.
- Adds new external network endpoints.
- Changes ACL rules, firewall rules, or service principal grants.
- Adds new fields to logs or support bundles (PII risk).

17.2 Security review checklist

- Cryptographic primitives from System.Security.Cryptography — never hand-rolled.
- Secrets never logged. [DoNotLog] on every secret-bearing field.
- Constant-time comparison for security-sensitive equality (CryptographicOperations.FixedTimeEquals).
- Random values from RandomNumberGenerator, never System.Random.
- Network calls go via CorrelationDelegatingHandler (auto-mTLS in production).
- New JWT claims documented; nonce/jti included for replay protection.
- Threat model updated if new trust boundary introduced (SAD Appendix I).
- Tamper-negative test added if signature / hash logic changed.

17.3 No-go list

- MD5, SHA-1, RC4, DES, 3DES — banned. SHA-256 minimum.
- ECB mode — banned. GCM or CBC + HMAC.
- XML external entities (XXE) — banned. Use DtdProcessing.Prohibit.
- HTTP (no TLS) — banned for any external endpoint.
- Certificate validation disabled (ServicePointManager.ServerCertificateValidationCallback = (..) => true) — banned.

18. Onboarding Checklist (New Engineer)

18.1 Day 1 — accounts and access

- Azure DevOps account; added to ePACS project group.
- Git access; SSH key registered; signed-commits configured.
- Azure Key Vault access (read-only) for dev test certs.
- VM provisioning rights for personal dev environment.
- Slack / Teams channel: #epacs-engineering, #epacs-ops, #epacs-incidents.

18.2 Day 1 — environment

- Run scripts/onboarding/setup-dev-env.ps1 (installs Node, Python, Docker, etc.).
- Pull repo; run dotnet build; expect green.
- Bring up local stack via docker compose; run dotnet test --filter Category!=Integration.
- Provision a Hyper-V dev-pacs VM via scripts/dev/new-pacs-vm.ps1.

18.3 Week 1 — reading

- This guide (Engineering Guide) — full read.
- Solution Architecture & Design (ePACS_SAD_v1.2.docx) — full read.
- Implementation Plan v3.5 — at least Sections 1–3, 12, 21–24.
- ADRs 0001–0020 — all read; understand each decision.
- Pair with a buddy on a real PR review during Week 1.

18.4 Week 2 — first commits

- Pick a starter ticket tagged "good-first-issue".
- Open PR following Section 12 conventions; reviewers will guide.
- Run scripts/dev/verify-idempotency.ps1 on your branch — both before and after PR feedback.

18.5 Month 1 — ramp-up

- Shadow an incident response (Section 15 playbooks).
- Participate in DR drill (quarterly cadence).
- Author at least one ADR for any non-trivial decision.
- Present a 15-min walkthrough of one subsystem to the team.

19. Final Reminder — Critical Rules

These ten rules are repeated here intentionally. They are the most-violated rules in our incident history. Print this page; pin it next to your monitor.

1. Never hardcode paths, ports, thresholds, or credentials. Everything from configuration.
2. Every DELETE must write to sync_outbox in the same transaction (before-state captured).
3. Every amendment must include reason + approver (configurable enforcement).
4. AUTO_INCREMENT seed = pacsid × 10^{10} — never manually set AUTO_INCREMENT.
5. Every state transition writes a checkpoint (fsync'd, write-then-rename).
6. PII never appears in logs — use [Sensitive], [DoNotLog], [Mask] attributes.
7. Heartbeat / sync failures never block business operations — fire-and-forget with circuit breaker.
8. Schema migrations classified by DDL algorithm — COPY on > 1M rows uses pt-online-schema-change.
9. Backup before every destructive operation — mandatory, not optional.
10. Test with power-cut simulation — every long operation must survive hard shutdown.

When in doubt: ask. When still in doubt: open a draft PR and tag the team. Code that is unsafe to ship is never saved by being clever; it is saved by being reviewed.

End of Engineering Guide — v1.0